OBS Player Architecture

DOCK & STREAMER - ARCHITECTURE AND DATA FLOW

Project Catalyst Fund 11
Milestone 4 - Marketplace Updates & API Launch
Reporting period: 2026-06-23 to 2026-08-31
Issued: August 31, 2026

Prepared by Awen LLC · https://sync.land · Project #1100272

*How the **dock** (frontend) and the **streamer API** (backend) fit together to license, stream, and audit music playback during a live broadcast.*

TL;DR

The Sync.Land OBS Player is two components working together: a **dock** (the frontend SPA loaded as a browser panel inside OBS Studio) and a **streamer API** (the Personal-Access-Token-authenticated backend that hands it playlists, per-track license clearance, and stream URLs). The dock is what the streamer sees. The streamer API is what makes it work. Both live under one system called *Sync.Land OBS Player*.

This system is the **reference external application** that satisfies M4 acceptance criterion #3: "*Test case: an external application is able to verify a sync license via public API.*"

1 · Two words, two meanings: pinning them down

Both of the words in the project name overload two concepts. Worth stating explicitly so nothing gets confused later in this document.

Dock

- **Meaning 1 (OBS's built-in feature):** the *Custom Browser Dock*, a panel type inside OBS Studio that loads any URL you give it. Any OBS user can add one from the Docks menu.

- **Meaning 2 (this project's frontend):** the *Sync.Land Dock SPA*, the frontend web app hosted at `sync.land/dock/` that a streamer loads *into* that OBS panel. This is the software delivered by this project's frontend repo.

Streamer

- **Meaning 1 (the end user):** the person broadcasting on Twitch, YouTube Live, or Kick, the audience for the whole system.
- **Meaning 2 (the backend namespace):** the `/streamer/*` REST API namespace on Sync.Land, the endpoints that serve the dock. Named for who they're for.

Personal Access Token (PAT)

Format `sk_syncland_<40 characters>`, 53 characters total. Server stores SHA-256 hex only; plaintext is shown to the streamer exactly once at creation. Bearer credential for every dock → API call. Streamers mint their own at `sync.land/account/tokens/` and revoke them there.

SLFS-v1

Sync.Land Free Sync License, version `SLFS-v1.0-2026-07-11`. The licensing tier that permits creator-scale live-stream use with required attribution. Every track played through the dock is deemed licensed under it (unless the streamer has upgraded to Commercial Sync on specific tracks).

Attribution overlay

When the dock is loaded as an OBS *Browser Source* rather than a Custom Browser Dock, it strips its full UI to just the current attribution string, for example:
`Music: Cullah, via Sync.Land, sync.land/song/parietals`. The streamer places this on their scene layout so the on-air attribution meets SLFS-v1 §6.

---

2 · The two halves at a glance

The dock: frontend

A static Vite-built single-page app. Runs in the browser context that OBS Studio embeds in its Custom Browser Dock panel (or Browser Source layer).

- Hosted at **`sync.land/dock/`**
- Holds no secrets; ships as static HTML + CSS + JS (~21 kB gzipped)
- Receives a Personal Access Token from the streamer via form input or `?token=` URL
- Stashes the PAT in `localStorage`, Bearer-auths every backend call
- Delivered from the public open-source repo **`Awen-online/syncland-obs-player`** under **Apache 2.0**

The streamer API: backend

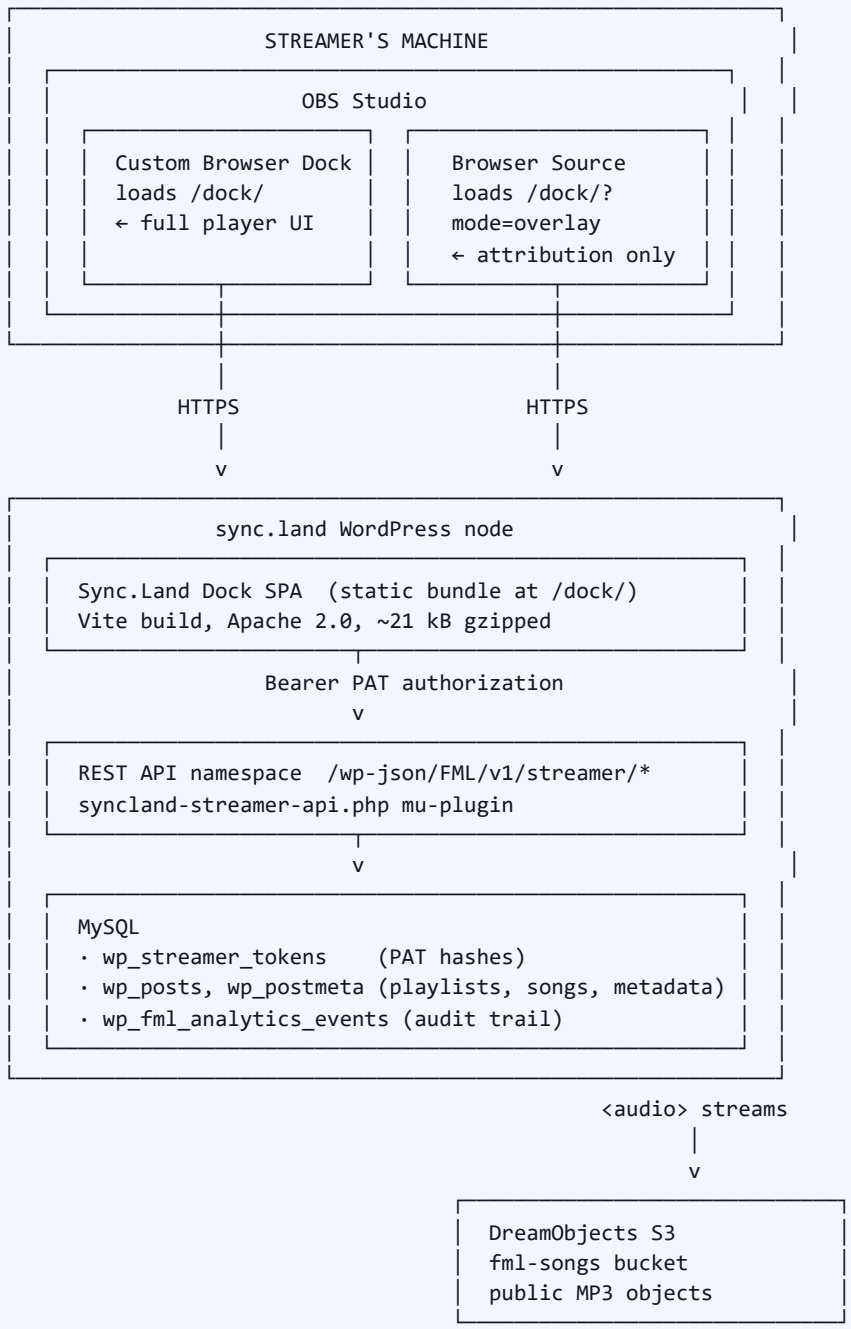
A REST API namespace at `/wp-json/FML/v1/streamer/*`, implemented as the `syncland-streamer-api.php` mu-plugin on the Sync.Land WordPress node.

- Validates the PAT against `wp_streamer_tokens` (SHA-256 hex lookup)
- Resolves the caller to a WordPress user, applies per-user data scoping
- Returns only that user's playlists / their songs' SLFS-v1 clearance
- Every response is CORS-open (PAT is the security boundary, not origin)
- Delivered from `Awen-online/sync-land` (private production dev repo), open-cored to `Awen-online/sync.land` (this repo) for Catalyst submission

---

3 · System components

Five moving parts. Two are inside OBS on the streamer's machine; three are on Sync.Land infrastructure.



4 · Data flow: provisioning a Personal Access Token

One-time, out of band. Streamer creates a PAT at `sync.land/account/tokens/`. The dock never sees this flow: it starts life with a PAT already in hand.

```
1.  Streamer         → sync.land/account/tokens/
                       "Generate token" button clicked
2.  WordPress node   → random_bytes(30), base64url-encode,
                       prefix "sk_syncland_" → 53-char token
3.  WordPress node   → INSERT into wp_streamer_tokens (
                       SHA-256 hash, prefix, name, user_id )
4.  WordPress node   → Return plaintext PAT to browser
                       (shown to streamer EXACTLY ONCE)
5.  Streamer         → Copy plaintext, paste into dock's PAT field
                       Server-side never retransmits plaintext
```

Recovery model. If the plaintext is lost, the streamer mints a new one. If compromised, the streamer revokes it by the public 16-char `prefix` (visible in the account UI) or by row `id`.

---

5 · Data flow: runtime cycle

Every OBS session that uses the dock follows this shape: sign-in, playlist load, per-track playback loop with license check and audit logging.

```
1.  OBS              → Load https://sync.land/dock/
                       (Custom Browser Dock or Browser Source)
2.  Dock SPA         → Read PAT from localStorage
                       (or show sign-in screen if empty)
3.  Dock SPA         → GET /streamer/me
                       Authorization: Bearer sk_syncland_...
    API              → { user_id, display_name, artist_ids }
4.  Dock SPA         → GET /streamer/playlists
    API              → { playlists[], tracks{} }
                       Each track: song_id, title, artist,
                                   duration, cover_url
5.  Streamer         → Picks a playlist from the picker screen

┌─── LOOP: each track ──────────────────────────────────┐
│
│ 6.  Dock SPA       → GET /streamer/track/{id}/clearance
│     API            → { can_stream, tier: SLFS-v1,
│                        attribution_text, stream_url,
│                        song: {album, duration, cover_url} }
│
│ 7a. IF can_stream:  Dock SPA renders attribution overlay,
│                     opens HTMLAudioElement against stream_url
│                     (DreamObjects S3 public MP3 object)
│ 7b. Dock SPA       → POST /streamer/track/{id}/played
│     API            → INSERT stream_play into
│                       wp_fml_analytics_events
│
│ 8a. IF NOT can_stream: Skip to next track. Reasons include
│                       no_license, artist_removed, artist_paused
│                       (never plays on air, attribution safety)
│
└──────────────────────────────────────────────────────┘
```

Every track is license-checked immediately before playback. A track that fails the check is skipped, never played, so a paused or removed track can never end up on air.

---

6 · API surface

All endpoints live under `/wp-json/FML/v1/streamer/*`. The dock uses the top four; the bottom three exist for the same-origin `/account/tokens/` account UI to mint and revoke tokens.

METHOD	ENDPOINT	AUTH	PURPOSE
GET	`/streamer/me`	PAT	Sanity-check the token, return caller identity.
GET	`/streamer/playlists`	PAT	List the streamer's playlists with track lists, cover art, duration per track.
GET	`/streamer/track/{id}/clearance`	PAT	Per-track license check + stream URL + attribution + album + duration + cover.
POST	`/streamer/track/{id}/played`	PAT	Log a play event to the audit trail (fire-and-forget).
GET	`/streamer/tokens`	WP nonce	List existing PATs (same-origin only, for the account UI).
POST	`/streamer/tokens`	WP nonce	Create a new PAT.
DELETE	`/streamer/tokens/{id}`	WP nonce	Revoke a PAT.

---

7 · Security model

- **PATs are stored server-side as SHA-256 hex only.** The plaintext is shown exactly once at creation, then unrecoverable.
- **Every PAT row carries a public 16-char prefix** (`sk_syncland_abcd`) so the streamer can identify and revoke a specific one without seeing the whole token again.

- `/streamer/*` **responses reflect the caller** `Origin` via `Access-Control-Allow-Origin`. That's safe because the PAT is the security boundary, origin restriction adds no real defense against an attacker who holds a valid token, but does block legitimate self-hosted dock mirrors from being useful.
- **The SPA holds the PAT only in browser** `localStorage`, scoped to the dock's origin. It is never posted to any endpoint except the Sync.Land API. Revoking a token invalidates it server-side even if the browser still has the plaintext.
- **Token-management endpoints (** `/streamer/tokens` **)** are **same-origin only** (WP nonce auth). A stolen PAT cannot mint or revoke other PATs, only the account UI running under a live `sync.land` session can.
- **No session cookies, no server-side per-request state, no OAuth complexity.** v0.2 will add OAuth2 / PKCE as an alternative for streamers who prefer not to handle tokens directly.

---

8 · What this proves for M4

M4 (*Marketplace Updates & API Launch*, 20,000 ADA) has four approved acceptance criteria. This system contributes concrete, machine-verifiable evidence to three of them.

M4 ACCEPTANCE CRITERION	WHERE THIS SYSTEM PROVES IT
Test case: an external application is able to verify a sync license via public API	Every `GET /streamer/track/{id}/clearance` call is exactly this, a fully external OBS instance receives a machine-readable SLFS-v1 clearance response with tier, attribution text, and stream URL. The dock demonstrates the flow end-to-end on air; the demo video captures the demonstration.
Continuous API optimization post-launch	v1.1 (2026-07-13) shipped the initial `/streamer/*` namespace. v1.2.0 (2026-07-15) shipped in-window enrichments: playlist response now includes `cover_url` + per-track `duration` + playlist-level cover; clearance response now includes `album`, `duration`, `cover_url` on the song object. CORS added to enable third-party dock hosts. All backward-compatible additions.
Ongoing documentation	This document + `SyncLand_API_Documentation_M4.pdf` + `docs/api.md` and `docs/streamer-setup.md` in `Awen-online/syncland-obs-player` + the mu-plugin's inline REST docstring + this pack's live interactive artifact.
User-feedback mechanisms	Every dock playback session writes a `stream_play` row to `wp_fml_analytics_events` with `via=obs_player`, a usage signal separate from marketplace-browsing telemetry. Feeds directly into the M4 User Feedback Report.

9 · Companion resources

- **Full API documentation:** `SyncLand_API_Documentation_M4.pdf`: API surface, PAT auth model, DB schema, reproducibility steps
- **OpenAPI specification:** `api-spec.yaml` at this repo's root, version **1.2.0**
- **Frontend source code:** `Awen-online/syncland-obs-player` (public, Apache 2.0)
- **Backend mu-plugin:** `syncland-streamer-api.php` in `Awen-online/sync-land` (private production repo)
- **Live production endpoints:**
  - Dock SPA: `https://sync.land/dock/`
  - Streamer API: `https://sync.land/wp-json/FML/v1/streamer/*`
  - Account UI: `https://sync.land/account/tokens/`

Prepared for Sync.Land / Awen LLC · Cardano Project Catalyst Fund 11 · Project #1100272

10 · v0.2.0: 2026-08-17

`Awen-online/syncland-obs-player` commit `b522ae9`, tagged **`v0.2.0`** and retrievable at `https://github.com/Awen-online/syncland-obs-player/releases/tag/v0.2.0`. The build serving `sync.land/dock/` is this commit. The v0.1 scaffold described above is superseded in three material ways.

10.1 Audio routing: the substantive change

A Custom Browser Dock is OBS *interface*, not a Source. Its audio goes to the system output device and never reaches the OBS mixer. It only reaches a stream through a Desktop Audio capture, which also carries notifications and every other system sound, and offers no per-source fader, filter or monitoring.

A **Browser Source**, by contrast, gets a real mixer channel.

So when an overlay Browser Source is present, it owns the `<audio>` element and the dock drives it remotely:

```
dock (interface)          overlay (Browser Source)        OBS
  queue                     <audio> element                mixer channel
  licence check    ──cmd──▶ play / pause / seek / volume    fader
  transport UI     ◀─state─ position / duration / ended      filters
                                                            monitoring
```

Transport travels over a `syncland-audio` BroadcastChannel on the shared origin. The overlay beacons every 1.5s; the dock hands audio over mid-song on the first beacon and reclaims it if beacons stop for five seconds, so the dock still works standalone. **Only one context ever holds a source**, both playing produces doubled, phase-offset audio.

Track advance is driven by the overlay's `ended` event, so the queue follows whatever is actually making sound.

10.2 Playback singleton

Playback moved out of the player screen into `playback.js`, owned by the app shell. Previously the screen owned the audio element, so navigating back tore the UI down while a detached `Audio` kept playing with nothing attached to it. A persistent bar and an on-screen now-playing block both render from that one store.

10.3 Licence verification is now visible

Criterion 3 asks that an external application verify a licence via the public API. v0.2.0 makes that legible on camera rather than implicit:

```
● Licence verified                    HTTP 200 · 142ms
GET sync.land/wp-json/FML/v1/streamer/track/11907/clearance
Free Sync License
attribution required · verified Mon, 17 Aug 2026 06:04:46 UTC
[ Copy verification receipt ]
```

The receipt button copies request line, status, latency and the full JSON verdict, the "link to the successful API request" artefact as a paste.

10.4 Also in v0.2.0

Overlay lower third (cover art, track, attribution, brand mark) driven entirely by the dock so the Browser Source URL carries **no token**; five themes that reach the overlay live; a settings screen; fade in/out with a talk-over duck; an in-dock OBS setup panel with copy buttons; and `window.obsstudio` detection so the dock states plainly whether it is inside OBS or a browser tab.

10.5 Corrections to v0.1 documentation

- The v0.1 note advising *"right-click the source → Audio Monitoring"* is **wrong for a dock**. A dock is not a source and has no such menu.
- Overlay transparency never worked: CSS cleared `body` and `#app` but not `<html>`, so OBS composited a solid `#0e0e1a` rectangle over the scene.
- Production builds had been pinned to the **dev** API by a stray `.env.local`, which Vite ranks above `.env.production`.

10.6 Known limits

- Overlay audio handover is verified by construction and in a browser, **not yet end to end inside OBS with a live token**.
- Loudness normalisation ships **off by default**; it is RMS, not LUFS.
- Stream URLs are unsigned S3 links and now also travel to the overlay context.